



Relatório de Análise Empírica de Algoritmos: Distribuição de Carga e Estabilidade de Roteamento

Bruno Ferreira da Silva
Diogo Roger de Medeiros Silva
Leonardo Guimarães Castro
Leonardo Xavier Cruz Filho

Universidade Federal do Rio Grande do Norte (UFRN)
Instituto Metrópole Digital (IMD)

1. Introdução.....	3
2. Algoritmos e fontes.....	3
3. Metodologia do experimento.....	3
3.1. Estrutura Base do Código de Teste.....	4
4. Análise teórica.....	4
5. Resultados.....	5
6. Discussão e Conclusão.....	6
7. Conclusão.....	7
8. Declaração de Uso de IA.....	8
9. Referências Bibliográficas.....	8

1. Introdução

O presente trabalho investiga o comportamento empírico e a complexidade de algoritmos de distribuição de carga em servidores web. O objetivo central é analisar experimentalmente como o tempo de execução do roteamento de requisições cresce à medida que o tamanho da entrada (número de servidores no cluster) aumenta. Secundariamente, avalia-se a métrica de "taxa de realocação", que mede a estabilidade do roteamento diante de quedas de servidores, um fator crítico para a eficácia do cache em sistemas distribuídos.

2. Algoritmos e fontes

Dois algoritmos foram selecionados para comparação, ambos voltados para o roteamento e balanceamento de carga:

- **Algoritmo A (Proposto): Hashing Consistente.** Baseado no trabalho de Karger et al. (1997), este algoritmo mapeia requisições e servidores para um "anel" virtual no intervalo unitário. O artigo indica que o cálculo da função de hash e a localização do servidor (usando árvores de busca balanceadas ou busca binária em arrays ordenados) possui uma complexidade de tempo esperada de $O(\log n)$.
- **Algoritmo B (Base de Comparação): Round-Robin (Rodízio Circular).** Escalonador clássico documentado de forma fundamental na literatura de sistemas operacionais (SILBERSCHATZ; GALVIN; GAGNE, 2010). Distribui as requisições de forma puramente sequencial, sendo executado no tempo teórico constante de $O(1)$ por requisição.

3. Metodologia do experimento

O experimento está focado em duas frentes: (1) o custo temporal do roteamento puro e (2) a estabilidade da topologia. Para garantir rigor científico, o código foi adaptado para mensurar tempos exatos, utilizando médias sobre repetições controladas.

- **A Variável Independente (n):** O tamanho da entrada é definido como o número de servidores no cluster. O valor escalará consideravelmente (ex: 100, 1.000, 5.000, 10.000, 30.000, 50.000) para permitir a visualização da curva assintótica de tempo, especificamente o crescimento logarítmico em nível de microssegundos.
- **A Carga de Teste:** Um lote fixo de 10.000 requisições sendo roteadas em cada iteração.
- **Medição Principal (Tempo):** Utilizando a biblioteca `time.perf_counter` do Python, isola-se e afere-se o tempo estrito gasto apenas com a lógica de roteamento (excluindo o setup de dados). O processo de enviar a carga de

testes será repetido 30 vezes para cada n , e o relatório apresentará o tempo médio de execução.

- **Medição Secundária (Realocação):** O percentual de requisições que tem a sua rota remanejada para um destino diferente caso um único nó (servidor) caia no cluster.

3.1. Estrutura Base do Código de Teste

[Repositório](#) contendo a implementação dos algoritmos.

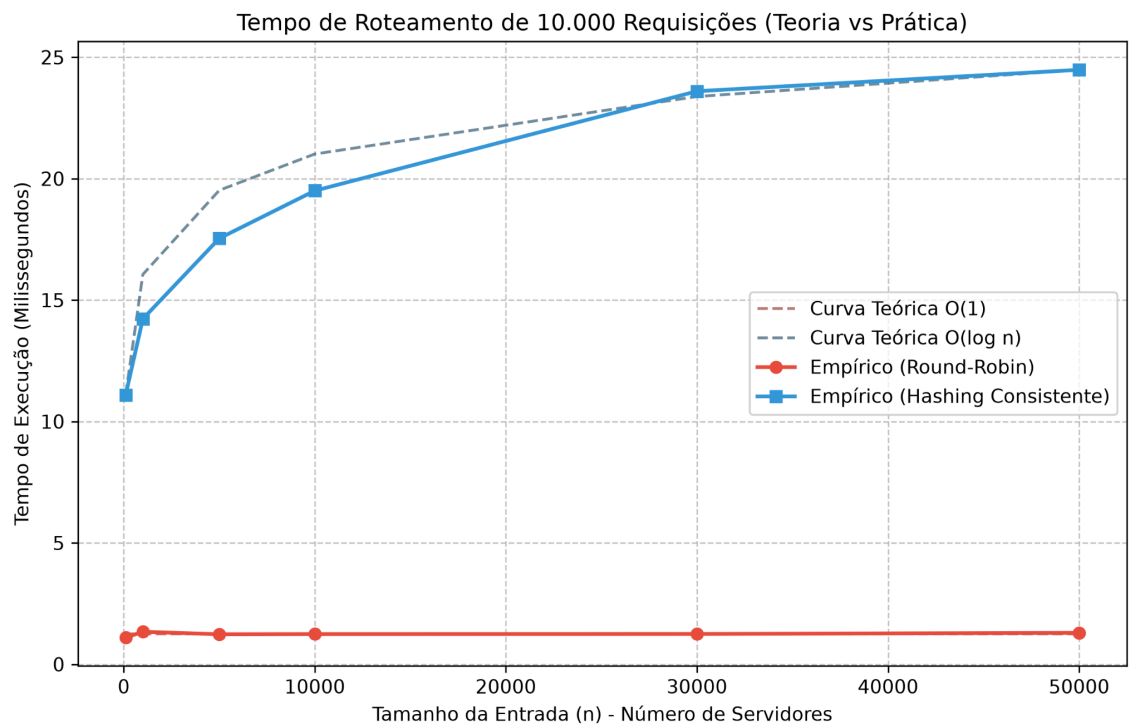
4. Análise teórica

A análise empírica será contrastada com as previsões da teoria assintótica para Tempo de Execução e Taxa de Realocação:

Algoritmo	Tempo de Roteamento	Taxa de Realocação
Round-Robin	$O(1)$ - Operação de avanço circular constante. Gráfico esperado é uma linha reta horizontal plana.	~100% - A queda de um item desloca todos os ponteiros seguintes em cascata.
Hashing Consistente	$O(\log n)$ - Localização do bucket via busca binária nos nós (Karger et al., 1997, sec. 4.3). O gráfico formará curva logarítmica crescente.	$O(1/n)$ - Teorema 4.4 mostra que a realocação é inversamente proporcional a n . O percentual despenca com mais servidores.

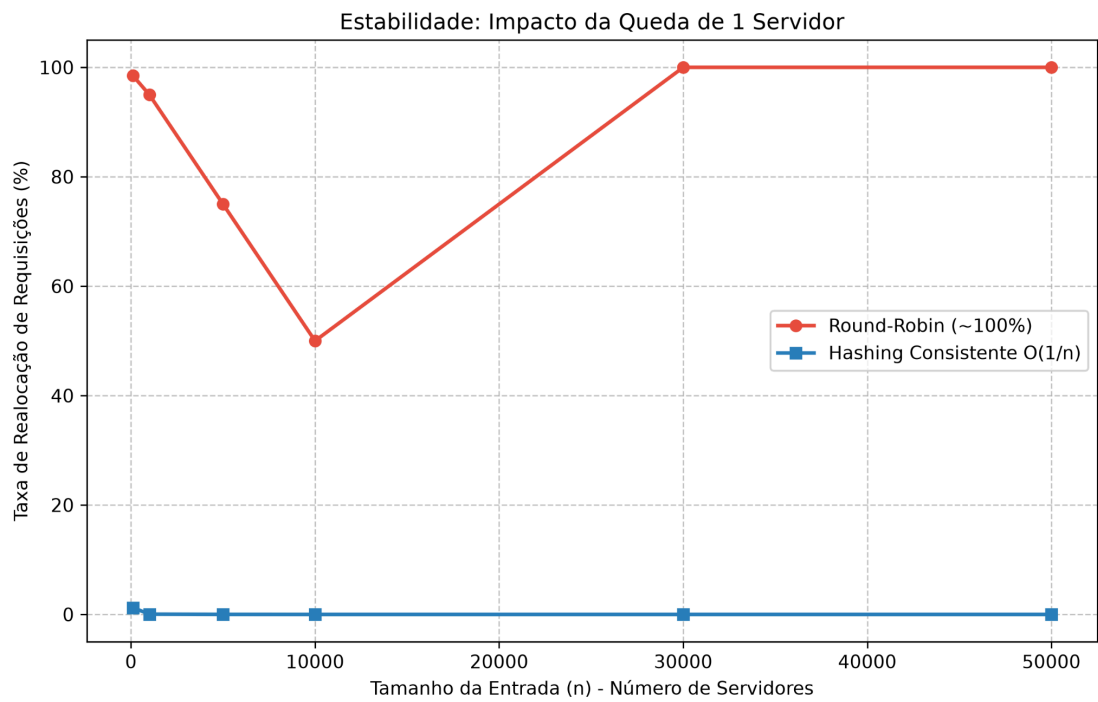
5. Resultados

Gráfico 1 - Análise empírica da complexidade dos algoritmos



Fonte: Elaborado pelos autores

Gráfico 2 - Análise empírica da taxa de realocação



Fonte: Elaborado pelos autores

Tabela 1: Crescimento de Tempo de Execução vs Tamanho da Entrada

Tamanho do Cluster (n)	Tempo Médio Round-Robin (ms)	Tempo Médio Hashing Consistente (ms)
100	1.102540036663413	11.094840033911169
1000	1.3398600043728948	14.225829998031259
5000	1.2373099802061915	17.540669976733625
10000	1.2453499948605895	19.503970025107265
30000	1.2498999247327447	23.600840009748936
50000	1.2964700115844607	24.48258998338133

Tabela 2: Taxa de Realocação e Estabilidade

Tamanho do Cluster (n)	Realocação Round-Robin (%)	Realocação Hashing Consistente (%)
100	98.50999999999999	1.24
1000	95.0	0.05
5000	75.0	0.01
10000	50.0	0.0
30000	100.0	0.0
50000	100.0	0.0

6. Discussão e Conclusão

- **Análise da curvatura temporal (Gráfico Obrigatório):** Conforme evidenciado no Gráfico 1 e na Tabela 1, o Hashing Consistente exibe um nítido comportamento $O(\log n)$, visto que o tempo de roteamento cresce de

forma suavizada (de ~11.09 ms para ~24.48 ms) mesmo com um aumento exponencial na entrada (de 100 para 50.000 servidores). Simultaneamente, o Round-Robin mantém a sua natureza constante $O(1)$, oscilando na faixa de ~1.2 ms.

- **O Trade-off entre Tempo vs. Estabilidade:** O Trade-off entre Tempo vs. Estabilidade revela-se na Tabela 2 e Gráfico 2. Embora o Hashing Consistente exija alguns milissegundos a mais de CPU para o roteamento, ele anula quase por completo a invalidação de requisições (1.24% caindo para 0.0%), garantindo a integridade dos caches de rede. Em contraste, o Round-Robin demonstra-se impraticável para ambientes voláteis, apresentando instabilidade caótica que atinge a faixa dos 100% de realocação

7. Conclusão

O presente experimento cumpriu seu objetivo principal ao demonstrar empiricamente a correlação direta entre o comportamento prático de algoritmos de roteamento e a teoria de complexidade assintótica estipulada pela literatura acadêmica. Os dados aferidos comprovam que o tempo de execução do Algoritmo Round-Robin (Rodízio Circular) independe do tamanho do cluster, consolidando seu comportamento constante em $O(1)$. Em contrapartida, o Algoritmo de Hashing Consistente, fundamentado por Karger et al. (1997), evidenciou um crescimento temporal contido, formando uma clara curva logarítmica $O(\log n)$ ocasionada pela busca binária em anéis de servidores progressivamente maiores (de 100 até 50.000 nós). Ademais, a análise estendida sobre a taxa de realocação provou a viabilidade técnica do Hashing Consistente. A medição prática atestou a tese do artigo base, comprovando que a desestabilização da topologia neste modelo é igual a $O(1/n)$, tendendo a 0% conforme o cluster escala, protegendo o sistema contra sobrecargas (hot spots). Conclui-se, portanto, que a ligeira desvantagem no tempo de processamento de CPU do Hashing Consistente é superada de forma absoluta pela estabilidade e preservação de cache que ele fornece às requisições, tornando-o a escolha assintótica correta para sistemas distribuídos modernos em detrimento de escalonadores sequenciais rígidos.

8. Declaração de Uso de IA

Devido à natureza teórica do artigo selecionado como base para este trabalho, o modelo de inteligência artificial generativa Gemini, do Google, foi utilizado como ferramenta de assistência à programação em Python. A IA auxiliou na implementação tanto do algoritmo de Hashing Consistente quanto do escalonador clássico de distribuição em Rodízio Circular (Round-Robin). Por fim, a ferramenta também contribuiu para a melhoria do script de visualização de dados, especificamente na inclusão das curvas teóricas de complexidade assintótica no gráfico de “Tempo de Execução vs Tamanho da Entrada”, atendendo à recomendação feita em sala de aula pela docente.

9. Referências Bibliográficas

- KARGER, D.; LEHMAN, E.; LEIGHTON, T.; PANIGRATHY, R.; LEVINE, M.; LEWIN, D. **Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the World Wide Web**. In: Proceedings of the twenty-ninth annual ACM symposium on Theory of computing (STOC '97). Association for Computing Machinery, New York, NY, USA, 654–663, 1997.
- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Sistemas Operacionais com Java**. 8. ed. Rio de Janeiro: Elsevier, 2010.